

HRIS Prototype — Walkthrough & Demo Guide

Who this is for: the person you're demoing to. Read this first, then use the "Demo script" at the bottom to walk them through it in ~7 minutes.

1. What is this prototype?

This is a **clickable, browser-only prototype** of the HRIS (HR Information System) described in `HRIS_Implementation.md`. It looks and feels like the finished product. Everything you can click responds, and the data is stitched together the way it will be in production — you can apply for a leave as an employee, switch to the HR admin's view, approve or reject it, and watch the employee's balance and notifications update in real time.

What is real in the prototype

- Every screen, every color, every animation from the spec
- The full leave workflow: apply → notify → approve/reject → balance updates
- Live attendance timer with clock-in / break / clock-out and overtime → replacement-leave conversion
- Notifications between employees and admins
- Holiday manager with in-app notice dispatch
- Analytics charts driven by the same data model

What is not yet real

- No database — data resets when the browser tab is closed (except the logged-in user, which persists via `localStorage`)
- No real email — the "Email" channel is shown in the UI but doesn't leave the browser; in production it would use SMTP/SendGrid/Resend
- PDF exports show a toast instead of downloading — server-side rendering (Puppeteer / WeasyPrint) is planned for the real build
- Biometric attendance — the API is designed for it (see spec §15), but this demo uses the manual clock-in button

Why prototype first, not full build? This lets your stakeholder *feel* how the product works before we invest in the backend, database schema, auth system and email infrastructure. If anything about the flow needs to change, changing it here takes minutes. Changing it after the backend is built takes days.

2. The two roles you'll be switching between

Role	Demo account	What they see
Employee	Nazmul Hasan (nazmul@company.com)	Personal dashboard, leave application, attendance timer, own analytics
Admin (HR)	Fatima Khan (fatima@company.com)	Everything above, plus leave inbox, employee directory, holiday manager
Admin (CEO)	Rafiq Ahmed	Same access as Fatima
Employees for context	Priya, Arif, Mira, Tahmid	So the admin views have real names to review

How to switch: top-right of every page there's a "Demo: switch user" chip. Click it, pick a name, and the whole app renders as that person. No need to log out. This is a demo-only shortcut — in production you'd sign in normally.

3. Screen-by-screen: what does what

3.1 Login page (/login)

- Left panel: brand and marketing. Right panel: sign-in card.
- Any email that matches a demo account signs in — the password field is a visual placeholder. Below "Demo accounts" you can one-click into any user.
- **In production:** JWT auth, "Forgot password" wired up, no self-signup (admins create accounts).

3.2 App shell

- **Sidebar** (left, 248 px). Two navigation sections: *Workspace* (visible to everyone) and *Administration* (only visible to admins).
- **Top bar** (72 px tall). Contains:
 - Personalized greeting + today's date on the left
 - "Demo: switch user" chip (only in the prototype)
 - Notification bell with unread badge — click for a dropdown
 - Avatar menu with account settings and Sign out
- **Toast host** (top-right). Confirms every action ("Leave request submitted", "Clock in", "Holiday notice sent", etc.) and auto-dismisses.

3.3 Employee — Home (/)

- **Greeting** headline and "Apply for Leave" primary button.
- **Attendance widget** (blue gradient card) — the live clock. States:
 - *Not clocked in:* single "Clock In" button

- *Working*: live counting-up timer, "Start Break" and "Clock Out"
 - *On break*: main timer greys out, break sub-timer starts, "Resume Work" button
 - *Clocked out*: summary of worked / break / overtime, "See you tomorrow!"
- **Leave balances** — three arc-ring cards (Casual, Sick, Replacement). The rings animate on mount and after any decision. Each shows Used, Pending and Total below.
- **Recent leave activity** — last 4 requests with status pills.
- **Right rail**:
 - "Next holiday" tile
 - "You've earned N replacement day(s)" tip
 - Mini month calendar with colored status dots
 - "This month, at a glance" summary

3.4 Employee — Apply for Leave (modal)

Opened from the "Apply for Leave" button. A 5-step wizard:

1. **Type** — three cards (Casual / Sick / Replacement). Each shows days remaining. Cards with 0 balance are disabled.
2. **Dates** — start + end date pickers with a live duration counter ("You are applying for X working days"). If you overlap an existing request, an inline error appears. Half-day toggle appears when start = end.
3. **Details** — reason (required, 100 chars), description (optional, 500 chars), file attachment (pdf/jpg/png).
4. **Send** — channel picker: Email, In-app, or both. Recipients are auto-populated from the admins list.
5. **Review** — an auto-generated email message is shown, fully editable. Click "Reset to default" to restore the template.

Submitting shows a checkmark-draw animation and a "Your leave request has been submitted!" confirmation. Behind the scenes:

- A `PENDING` request is added to the store
- Admins get an in-app notification badge
- The employee's *pending* balance increases (so they can't overbook)

3.5 Employee — My Leaves (/leaves)

- Tabs: All / Pending / Approved / Rejected / Cancelled with live counts.
- Table shows type, period, duration, status, applied date, decided date.
- Only **Pending** rows show a "Cancel" action (returns the pending balance).
- Click any row to open a detail modal with the full request and any admin note.

3.6 Employee — Attendance (/attendance)

- Same attendance widget as home + a mini calendar.
- Four stat cards: Present, Absent, Leaves taken, Total overtime.
- Two charts: **Daily hours worked** (line) and **Daily overtime** (bar).
- Full daily breakdown table with clock-in, clock-out, break, worked, overtime, status.

3.7 Employee — Calendar (/calendar)

- Full-page monthly grid. Each day cell shows a colored dot (Present / Leave / Holiday) and a label.
- Legend below. "Upcoming" section lists the next 5 holidays.

3.8 Employee — Analytics (/analytics)

- Four hero stats: **Attendance rate**, **Leave utilization**, **Overtime bank**, **Absences**.
- Charts: Leave distribution donut, Working hours area chart, Cumulative leaves line chart.
- Deliberately jargon-free ("no jargon" tagline).

3.9 Employee — Reports (/reports)

- Grid of downloadable report cards (Attendance / Leave / Summary; admins also see All-employees).
- Clicking "Download PDF" shows a toast explaining the production behavior.

3.10 Admin — Home (/admin)

- Greeting for the admin plus "Open leave inbox" CTA.
- **Four stat cards**: Pending Requests, Present Today, Approved this cycle, Next holiday.
- **Pending leave requests** panel — shows up to 5, click any row to open the review drawer.
- **Upcoming holidays** — each row has a "Send" button for a one-click in-app notice dispatch.
- **Leave usage by type** donut and **Leave usage by department** bar chart.

3.11 Admin — Leave Requests (/admin/requests)

- Full inbox: searchable by name / reason / department, filterable by status and by leave type.
- Table shows employee (with avatar), type, period, duration, reason, applied time, status.
- Clicking a row opens the **Review Drawer** from the right (480 px wide, full-screen on mobile). The drawer shows:
 - Employee card
 - Requested period
 - Balance preview: "Current balance" vs "If approved"
 - Reason, description, attachment
 - Notification channels used
 - Note field (optional)
 - Reject / Approve action bar
- Approve/Reject each pop a confirmation modal (Reject requires a reason) so nothing happens accidentally. On confirm:
 - Employee gets a toast + in-app notification
 - Their balance is deducted (Approve) or the pending amount is released (Reject)

3.12 Admin — Employees (/admin/employees)

- Grid of employee cards. Each shows avatar, name, role, department badges, employee ID, email, and the three current balances (casual / sick / replacement).
- Searchable.

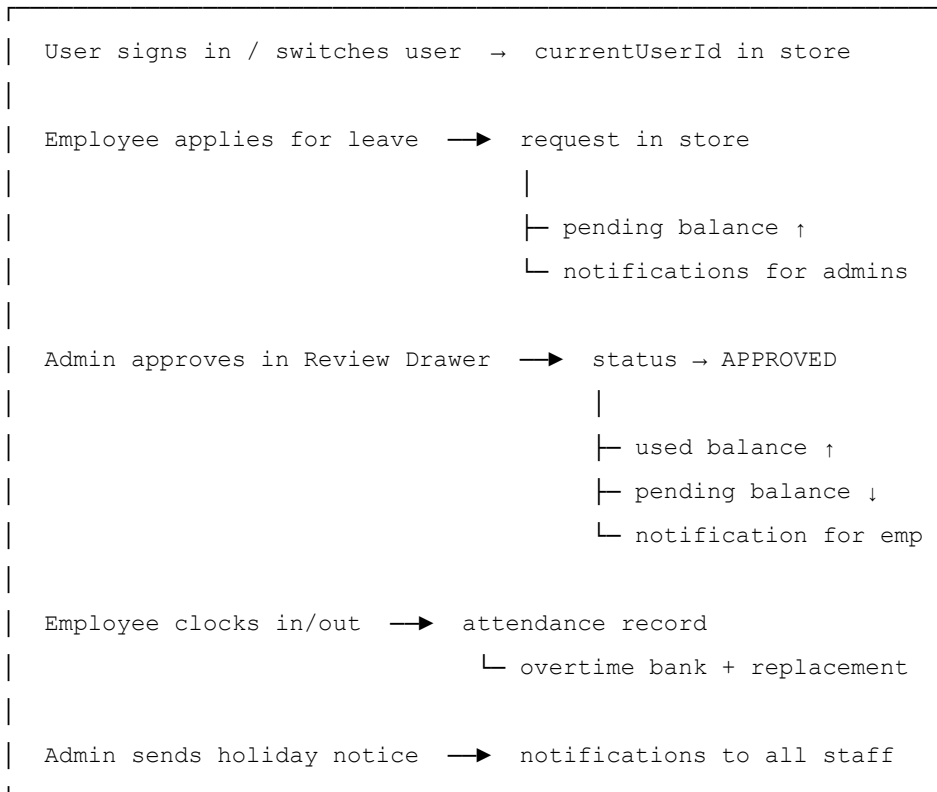
3.13 Admin — Holiday Manager (/admin/holidays)

- List of all holidays with a colored date tile.
- Actions per row: Send notice (if unsent), Edit, Delete.
- "Send notice" shows a preview of the exact message before sending. On confirm, every non-super-admin gets an in-app notification and a toast confirms delivery.
- "New holiday" opens a form (name / date / description / recipients).

3.14 Admin — Settings (/admin/settings)

- Read-only settings cards for the demo: working hours, leave policy, notifications, roles, and biometric integration note.

4. How the pieces connect



All state lives in a single Zustand (<https://github.com/pmndrs/zustand>) store at `src/lib/store.ts`. Actions like `submitLeave`, `approveLeave`, `clockIn`, `sendHolidayNotice` update the store — every screen reacts automatically.

The seed data lives in `src/lib/mockData.ts` — six employees, sample leave requests (some pending, some historical), four holidays, and one month of attendance for Nazmul.

When you swap this for a real backend, you replace the store actions with API calls (see the endpoint list in `HRIS_Implementation.md §11`). The UI does not change.

5. Demo script (7 minutes)

0:00 — Login

"This is HRIS — People, simplified. It's a web app for running employee leaves, attendance and holidays. I'm going to walk through both sides — what an employee sees, then what HR sees."

Click the **Nazmul** quick-pick card on the login page.

0:30 — Employee dashboard

"This is Nazmul's home screen. Three things I want you to notice:"

Point at each in turn:

1. The **attendance timer** at the top — live, shows today's session.
2. The **leave balance rings** — one for each leave type. They animate on load, and shrink as leave is used.
3. The **calendar and activity list** — everything Nazmul needs to see at a glance.

1:15 — Applying for leave

"Nazmul needs to take a couple of days off. Here's what that flow looks like."

Click **Apply for Leave**. Walk through each step:

- Pick **Casual Leave**
- Set start date `2026-08-27` and end date `2026-08-28`
- Reason: `Family event`
- Leave both **Email** and check **In-app** as channels
- Show the auto-generated review message; click **Edit** to prove it's editable
- Click **Submit Leave Request**

"Notice the checkmark animation and the toast at the top. Behind the scenes, HR just got notified."

3:00 — Switching to the HR view

"Now I'll switch to Fatima, our HR manager."

Click **Demo: switch user** in the top bar → pick **Fatima Khan**.

"This is the same app, but from HR's angle. She has an extra sidebar section, and the top card shows exactly how many leave requests are waiting for her decision."

3:30 — Approving the leave

Click **Open leave inbox**. Point out the filters and search.

Click Nazmul's new request. The **Review Drawer** slides in from the right.

"This is what HR sees when reviewing a request. Everything's in one place — the employee, the dates, the reason, and — this is important — Fatima can see what Nazmul's balance would look like if she approves it."

Add a note like "Enjoy your break!" then click **Approve** → confirm.

"Nazmul's leave is now approved. His balance updated, and he just got a notification."

4:30 — Back to the employee

Switch user back to **Nazmul**.

"There's the approved leave in his history, his balance is now down by 2 days, and there's an unread notification in the bell."

Click the **bell** to show the "Your leave request was approved" notification.

Open **My Leaves** to show the updated table.

5:15 — Attendance & analytics

Open **Attendance**.

"This is where working hours are tracked. Right now it's the browser button, but the same API accepts biometric fingerprint scanners — so when we install a scanner, no code changes are needed."

Show the charts and the daily breakdown table.

Open **Analytics**.

"And these are the analytics. No jargon. Attendance rate, how much leave is used, overtime accumulating toward replacement days, and clear charts."

6:15 — Holiday manager (switch back to Fatima)

Switch to **Fatima**. Open **Holiday Manager**.

"HR manages the company calendar here. Each holiday can auto-generate a nicely-worded notification. Watch this."

Click **Send notice** on **Eid ul-Fitr** → preview the message → confirm.

"That notice just went to every staff member as an in-app notification. In the production version this also sends an email using the company's standard template."

7:00 — Close

"That's the core loop. A few things worth mentioning:"

- The design system is one you can extend — same colors, spacing and typography everywhere.
- Every action is instant with no page reloads (we're using React, Zustand and Framer Motion).
- It's fully responsive — this works on tablet and phone too.
- Everything you saw is written against the spec in `HRIS_Implementation.md`, and the backend endpoints are already designed — we just haven't built them yet.

"Any questions?"

6. Things a curious viewer will ask

"Where does the data go?" Right now, everything lives in the browser (`localStorage` for the logged-in user; in-memory for the rest). Nothing leaves the machine. In production this is a PostgreSQL database (see spec §10 for the full schema) served over a REST API (spec §11).

"How does email work?" The UI already generates the exact email that would be sent. In production we plug in an SMTP server (SendGrid or Resend for hosted; Nodemailer for self-hosted). All sent emails are logged in the `email_log` table for audit.

"How long to build the real thing?" Because the design, flows, and schema are all defined, a working MVP with a real backend is roughly 4–6 weeks of engineering. Adding biometric scanners is another 1–2 weeks after the hardware arrives.

"Can we change X?" Yes — that's why we're prototyping first. Changes to the flow now cost minutes. After the backend is built, they cost days.

"Is it secure?" The spec calls for JWT auth with refresh tokens, HTTPS everywhere, and role-based access on every endpoint. This prototype doesn't implement auth because there's no server to authenticate against.

7. Deploying this to Vercel

The prototype is a static single-page app. Any static host works, but Vercel is easiest.

Option A — Vercel CLI (fastest)

```
cd hris
npm install -g vercel
vercel                # first time, follow prompts
vercel --prod         # promote to production
```

When asked, accept the detected framework (**Vite**) and defaults.

Option B — Vercel dashboard (via GitHub)

1. Create a GitHub repo and push the `hris/` folder to it. Example:

```
cd hris
git init
git add .
git commit -m "HRIS prototype"
git branch -M main
git remote add origin https://github.com/<you>/hris.git
git push -u origin main
```

2. Go to <https://vercel.com/new> (<https://vercel.com/new>) and import the repo.
3. If your repo has multiple folders, set **Root Directory** to `hris`.
4. Framework preset should auto-detect as **Vite**. Build command: `npm run build`. Output directory: `dist`.
5. Click **Deploy**. In under a minute you get a URL like `https://hris-abc123.vercel.app`.

Why `vercel.json` is in the repo

The prototype uses client-side routing (`react-router-dom`). Without a rewrite rule, visiting `/admin/requests` directly would 404 on Vercel. `vercel.json` rewrites every request to `/`, and React Router handles the rest.

Custom domain

In the Vercel project settings, add a custom domain (`hris.company.com`). Vercel gives you a CNAME target — add it in your DNS provider — done.

8. Sharing the prototype

Send the person you're demoing to:

- **The URL** from Vercel
- A one-liner: *"Sign in with any demo account on the login page. The 'Demo: switch user' chip in the top bar toggles between the employee and HR views."*
- Optionally, this document.

Because there's no backend, anyone with the URL can experiment freely — their changes only affect their browser.